

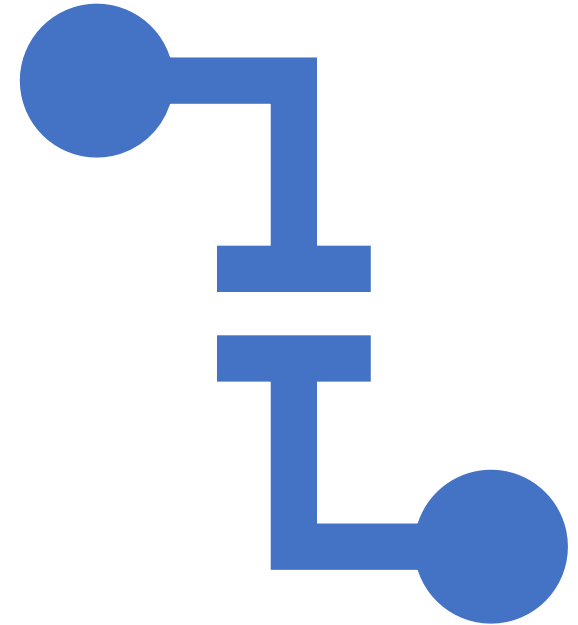
Software Architecture and Design Thinking 116U01C701

Module 2

Module 2: Connectors

2.1 Connector Foundations, Connector Roles

2.2 Connector Types and Their Variation Dimensions



Connectors

Purpose of Connectors:

- hold software components together
- Enable components to interact and communicate
- Facilitate data transfer, control flow, and synchronization between different parts of a software system

Connectors

Importance of Connectors:

Provide:

- **Modularity**
- **Reusability**
- **Scalability**
- **Flexibility**

Connectors

Modularity:

- They help to create **modular systems** by defining clear interfaces between components.
- Allows developers to work on individual components independently without needing to know the internal workings of other components.

Reusability:

- Well-designed connectors enable components to be reused in different contexts.
- A component can be "plugged into" different systems as long as the connector's interface is compatible.

Connectors

Scalability:

- Message-based connectors are essential for building **scalable systems**.
- Allow for the addition of new components or the replication of existing ones without major architectural changes.

Flexibility:

- Allow for the replacement or modification of a component without affecting the rest of the system, as long as the new component adheres to the same connector interface.

Connectors And Components

- **Components** and **Connectors** are two fundamental building blocks

Components

A component

- **Functionality:**
- Components are the "workers" of the system.
 - Perform computations, manage data, and execute business logic.
- **Encapsulated:**
- A self-contained, encapsulated unit of a software system that performs a specific function
 - They hide their internal workings from other parts of the system. You don't need to know how a component is implemented, only how to use its public interface.

Components

A component

- **Reusable:**
- A well-designed component can be used in different applications or in multiple places within the same
 - It's an independent, reusable piece of code that exposes its capabilities through a well-defined interface.

Connectors

A Connectors

- The communication mechanism that enables components to interact with each other. It defines the rules and protocols for how data and control flow between components.
- Connectors are the "managers" or "facilitators" of the system, not the workers.

Provides:

- **Communication:** Connectors are the communication channels. They transmit data, invoke methods, and coordinate the flow of information.

Connectors

A Connectors

- **Decoupling:**
 - Help reduce dependencies between components.
 - By mediating communication, allow components to be developed, updated, and deployed independently.
- **Protocol:**
 - Define the specific rules of interaction, such as whether communication is synchronous (like a procedure call) or asynchronous (like a message queue).

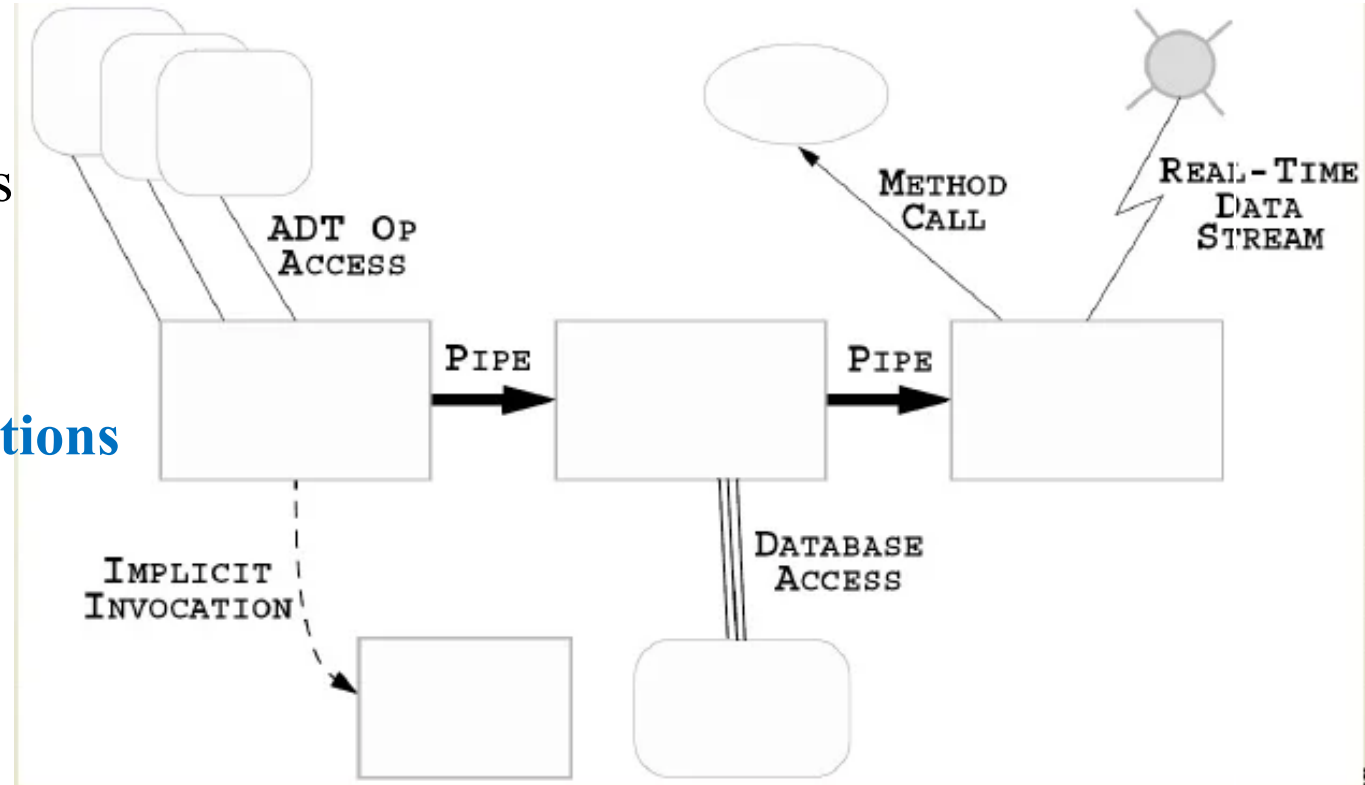
Components V/s Connectors

	Feature	Component	Connector
Role		Performs computation; provides functionality	Facilitates communication; manages interaction
Focus		Internal logic and data management	Communication protocols and data transfer
Implementation		A module, library, or microservice	An API, message broker, or communication protocol
Responsibility		"What to do"	"How to communicate"
Independence		Can be developed and tested in isolation	Relies on components to define communication endpoints
Analogy		A building	A road or a phone line

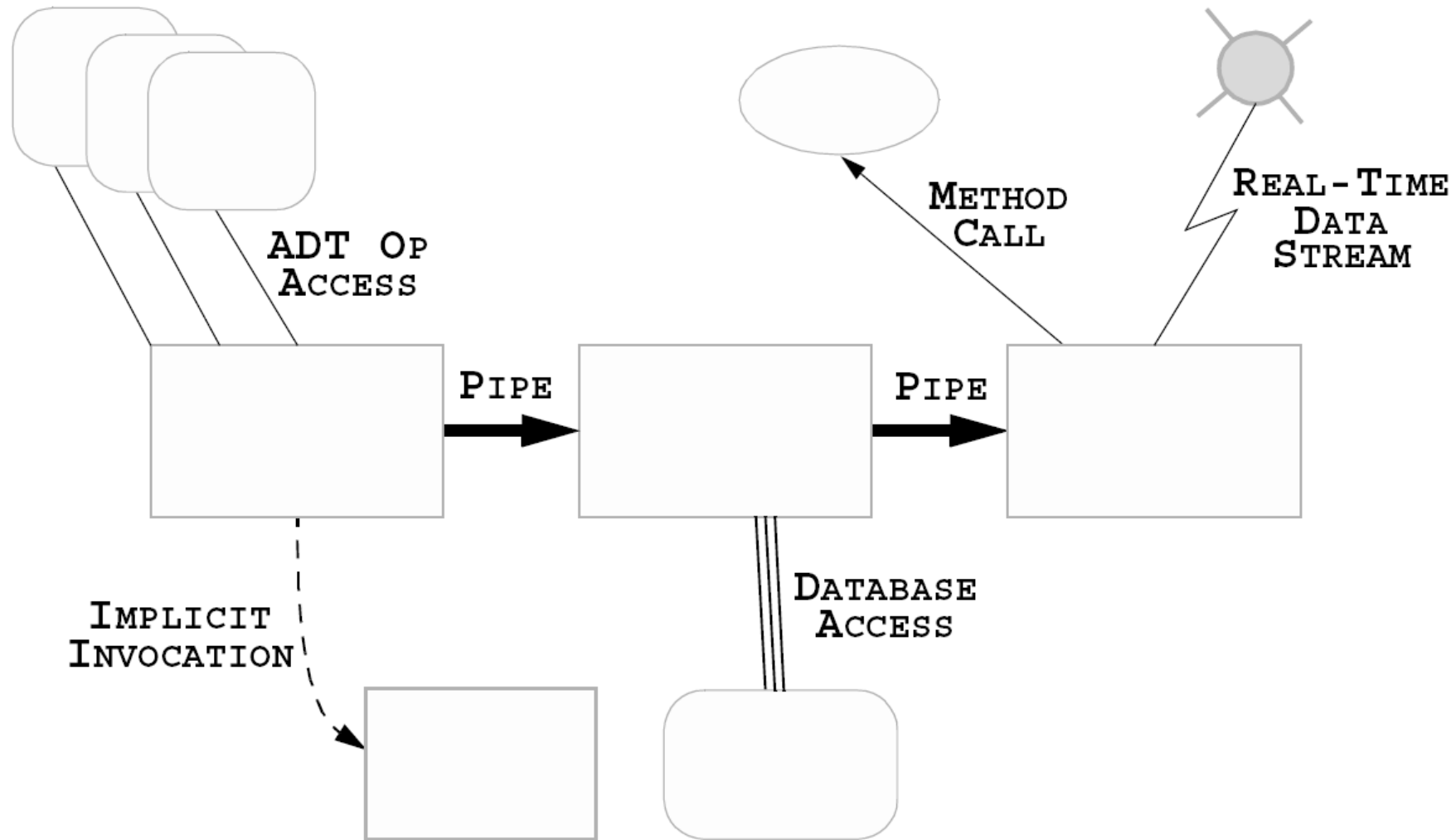
Connectors

What is a Software Connector?

- **Architectural element that models**
 - Interactions among components
 - Rules that govern those interactions
- **Simple interactions**
 - Procedure calls
 - Shared variable access
- **Complex & semantically rich interactions**
 - Client-server protocols
 - Database access protocols
 - Asynchronous event multicast
- **Each connector provides**
 - Interaction duct(s)
 - Transfer of control and/or data



Where are Connectors in Software Systems?



Conceptual Connectors

- A **conceptual connector** is a **first-class entity** in a system's architectural design, (treated as a distinct, independent, and crucial element—just as important as a **component**. Instead of being a hidden implementation detail, the connector is explicitly designed and analyzed in its own right)
- An abstract model that describes the rules, protocols, and mechanisms of interaction between components.
- Independent of any specific programming language or technology.
- Allow architects to reason about the system's structure and behavior without getting bogged down in implementation details.

Conceptual Connectors

- **Focus:** Defines the *how* of communication (e.g., synchronous, asynchronous, event-driven, client-server).
- **Purpose:**
 - Enable architectural analysis, design, and reasoning.
 - Help ensure consistency and provides a clear separation between the components' functionality and their interaction patterns.
- **Characteristics:**
 - Has a distinct identity in architectural models.
 - Describes the communication protocol abstractly.
 - Independent of implementation details.
 - Examples: "Pipe and Filter," "Message Bus," "Remote Procedure Call."

Implemented Connectors

- An implemented connector is the actual code and infrastructure that realizes a conceptual connector.
- It's the concrete, low-level implementation of the communication mechanism.
- An implemented connector often doesn't have a dedicated, single-unit representation in the codebase.
- Its functionality is frequently **distributed** across multiple code modules, libraries, and communication mechanisms.

Implemented Connectors

- **Focus:**
 - The concrete code and infrastructure that enables the communication.
- **Purpose:**
 - To make the abstract communication protocol a reality in a running system. It handles the details of data serialization, network protocols, error handling, and component invocation.

Implemented Connectors

- **Characteristics:**

- Often has no single, dedicated code file or module.
- Its functionality is spread across different parts of the system.
- Tied to a specific technology or language (e.g., a REST API implementation in Python, a Kafka message queue setup).
- Examples: An HTTP request in a web framework, a function call to a gRPC library, a publish/subscribe method call to a message broker API.

Conceptual v/s Implemented Connectors

Feature	Conceptual Connector	Implemented Connector
Level of Abstraction	High-level architectural design	Low-level code and infrastructure
Form	Abstract model, first-class entity	Concrete code, often distributed
Purpose	Architectural reasoning and analysis	System functionality and execution
Identity	A distinct entity in design diagrams	Often no single identity in code
Examples	"Event Bus," "Client-Server"	A Python library for a message queue, a gRPC service call

Reasons for Treating Connectors Independently

- Connector $\neq$ Component
 - Components provide application-specific functionality
 - Connectors provide application-independent interaction mechanisms
- Interaction abstraction and/or parameterization
- Specification of complex interactions
 - Binary vs. N-ary
 - Asymmetric vs. Symmetric
 - Interaction protocols

Treating Connectors Independently (cont'd)

- Localization of interaction definition
- Extra-component system (interaction) information
- Component independence
- Component interaction flexibility

Benefits of First-Class Connectors

- When a connector is a first-class entity, it allows architects and developers to:
- **Explicitly model interactions:**
 - The communication between components is not assumed; it's a defined part of the design.
 - You can see the "how" of the communication on a diagram, such as a **message bus**, a **pipe**, or a **client-server link**.

Benefits of First-Class Connectors

- When a connector is a first-class entity, it allows architects and developers to:
- **Analyze and reason about the system:**
 - By treating connectors as separate entities, you can analyze their properties, such as latency, throughput, and security, independently of the components they connect.
 - This helps in making informed design decisions.

Benefits of First-Class Connectors

- When a connector is a first-class entity, it allows architects and developers to:
- **Promote reusability:**
 - A well-defined connector can be reused across different parts of a system or in entirely new systems, just like a component.

Benefits of First-Class Connectors

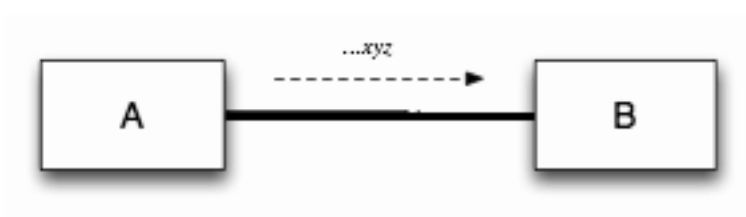
- When a connector is a first-class entity, it allows architects and developers to:
- **Prevent architectural erosion:**
 - It ensures that the implemented communication mechanisms in the code directly reflect the high-level design.
 - This prevents the architecture from becoming a mess of ad-hoc connections that don't match the original plan.

Making connectors first-class entities elevates their status from an invisible detail to a key architectural element, making the design more robust, understandable, and maintainable.

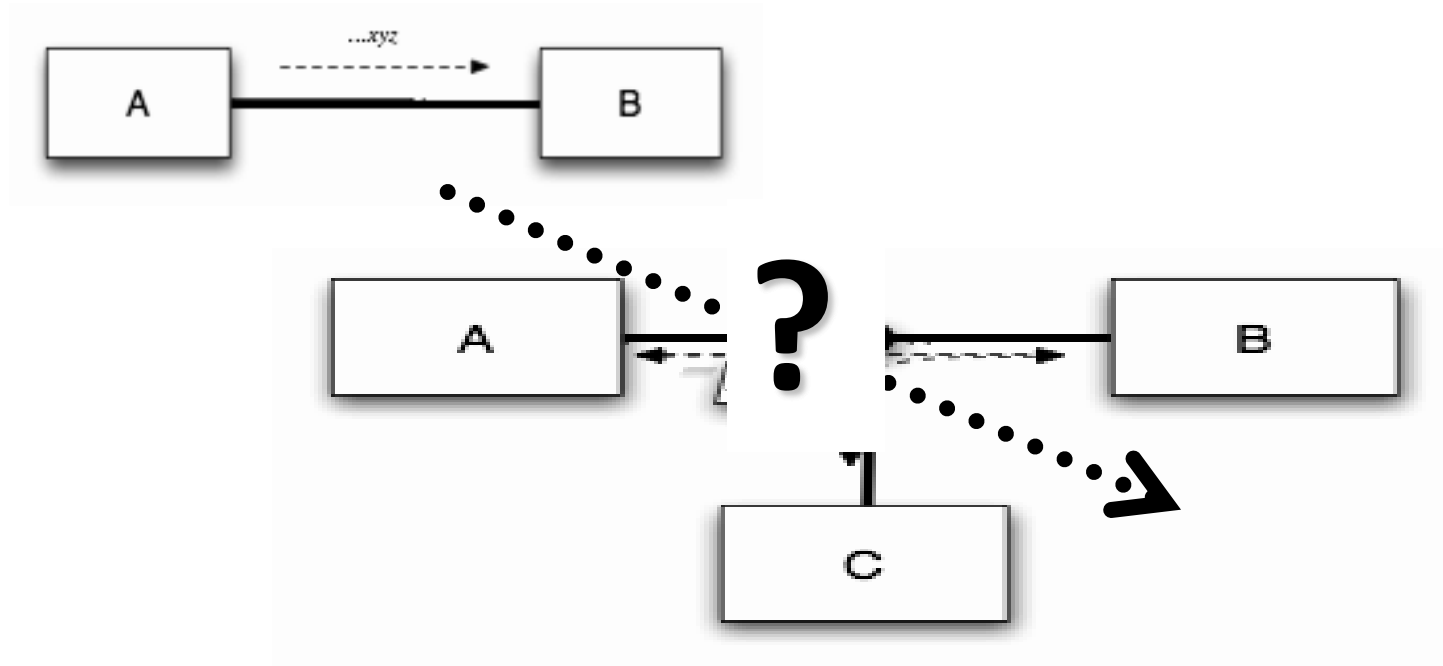
Benefits of First-Class Connectors

- Separate computation from interaction
- Minimize component interdependencies
- Support software evolution
 - At component-, connector-, & system-level
- Potential for supporting dynamism
- Facilitate heterogeneity
- Become points of distribution
- Aid system analysis & testing

An Example of Explicit Connectors



An Example of Explicit Connectors (cont'd)



Connector foundations

- Connectors mainly used for:
 - Flow of control: calling functions/ procedure or other programs
 - Flow of data: memory access
- Additionally:
 - Maintains one channel/ duct used to link interacting components
 - Support flow of data and control between them
- Simple connectors (module linker) provide services by forming duct between components
- Connector argument ducts with some combination of data and control flow will provide richer interaction services (connecting specific components based on the service needed)
- Very complex connectors can have internal architecture that includes computation and information storage.

Connector foundations

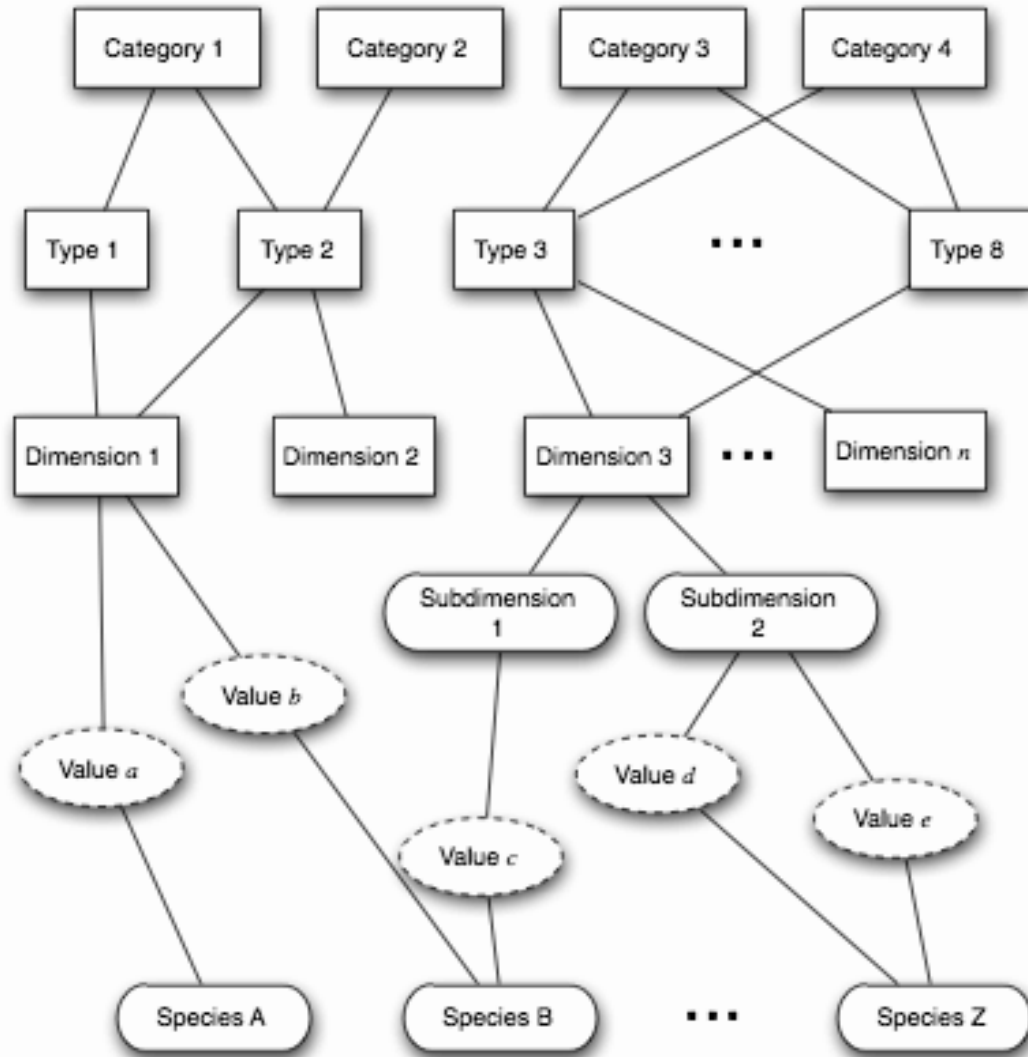
- **Simple connectors (module linker)**
 - Provide services by forming duct between components
 - Provide one type of interaction service
 - Implemented in programming languages

Connector foundations

- **Composite Connectors:**

- Connector argument ducts with some combination of data and control flow will provide richer interaction services (connecting specific components based on the service needed)
- Very complex connectors can have internal architecture that includes computation and information storage.
- Achieved through composition of several connectors (and possible components)
- Provided as libraries and frameworks
- Combine many kind of interactions
- Can help to overcome the limitation so f modern programming language
- Necessary to understand underlying, low-level interaction mechanism, identify appropriate design choice, direct potential mismatch among components

Framework for studying Connector



Category: Primary service / role provided

Type: way in which interaction services are realized

Dimension/ subdimension: architectural relevant details

Value: dimension instances

Species: values from different types

Software Connector Roles

- **Locus of interaction among set of components**
- **Protocol specification (sometimes implicit) that defines its properties**
 - Types of interfaces it is able to mediate
 - Assurances about interaction properties
 - Rules about interaction ordering
 - Interaction commitments (e.g., performance)
- **Roles**
 - Communication
 - Coordination
 - Conversion
 - Facilitation

Connectors for Communication

- **Main role associated with connectors**
- **Support:**
 - Different communication mechanisms
 - e.g. procedure call, Remote Procedure Calls, shared data access, message passing
 - Constraints on communication structure/direction
 - e.g. pipes
 - Used to pass messages, exchanges data to be processed and communicate results of computation

Connectors for Coordination

- Determine computation control mechanism
- Components interact by passing the thread of execution to each other
- Function calls and method invocation
- High order connectors such as signals and load balancing connectors provide richer, more complex interactions built around coordination services
- Control delivery of data

Connectors for Conversion

- Transform the interaction required by one component to that provided by another
- Enable heterogeneous components to interact
- Enable interaction of independently developed, mismatched components
- Mismatches based on interaction
 - Type
 - Number
 - Frequency
 - Order
- Examples of converters
 - Adaptors
 - Wrappers

Connectors for Facilitation

- Mediate and streamline components interaction
- Enable interaction of components intended to interoperate
- Govern access to shared information
- Ensure proper performance profiles
 - e.g., load balancing
- Provide synchronization mechanisms
 - Critical sections
 - Monitors

Connector Types/ levels

1. Procedure call
2. Data access
3. Event
4. Stream
5. Linkage
6. Distributor
7. Arbitrator
8. Adaptor

Features of Connectors

- Service(s) Provided
- Dimensions
- Sub- Dimensions
- Values
- **Service(s) Provided:**
 - Communication
 - Conversion
 - Coordination
 - Facilitation

Features of Connectors

- **Dimensions and sub-dimensions:**
- Different properties or characteristics used to classify, describe, and analyze its behavior and capabilities.
 - They provide a structured way to understand a connector's role and how it impacts the overall system architecture.

Connectors

The **dimensions**

- Define the high-level properties of their functionality.
- **Interaction:**
- Describes how components communicate through the connector.
 - For example, a connector can enable **procedure calls** (like a function call in a library) or **message passing** (like sending a message over a queue).
- **Arbitration:**
- Relates to how the connector manages conflicts or handles shared resources. A connector may use an **arbitrator** to control access to a shared resource, ensuring fairness and preventing deadlocks.

Connectors

The **dimensions**

- Define the high-level properties of their functionality.
- **Data Exchange:**
 - Concerns how data is transferred between components. Data can be passed **by value** (a copy of the data is sent) or **by reference** (a pointer to the data is sent).
- **Synchronization:**
 - Deals with the timing and coordination of component interactions. A connector can enable **synchronous** communication (the sender waits for a response) or **asynchronous** communication (the sender does not wait).
- **Protocol:**
 - Specifies the rules and conventions that components must follow to interact with the connector. It defines the specific data formats, message sequencing, and error handling mechanisms.
 - For example, a connector might follow a protocol like **HTTP** or **FTP**.

Connectors

The **dimensions**

- Define the high-level properties of their functionality.
- **Data Exchange:**
- Concerns how data is transferred between components. Data can be passed **by value** (a copy of the data is sent) or **by reference** (a pointer to the data is sent).
- **Synchronization:**
- Deals with the timing and coordination of component interactions. A connector can enable **synchronous** communication (the sender waits for a response) or **asynchronous** communication (the sender does not wait).
- **Protocol:**
- Specifies the rules and conventions that components must follow to interact with the connector. It defines the specific data formats, message sequencing, and error handling mechanisms.
 - For example, a connector might follow a protocol like **HTTP** or **FTP**.

Connectors

- **The Sub-dimensions**

- Provide a more detailed and granular view of each main dimension.
- Offer a deeper understanding of the connector's specific characteristics and how it's implemented.
- **Interaction Subdimensions:**
 - **Binding Time:**
 - When the connection is established.
 - It can be **static** (at compile-time) or **dynamic** (at runtime).
 - **Invocation Method:**
 - How the interaction is initiated.
 - Examples include **remote procedure call (RPC)**, **publish/subscribe**, or **event-driven** models.
 - **Topology:**
 - The structure of the connections, such as **point-to-point** or **broadcast**.
- **Arbitration Subdimensions:**
 - **Access Control:**
 - The rules governing which components can access a shared resource.
 - This can involve **security policies** or **privileges**.
 - **Scheduling:**
 - How the connector manages the order of requests.
 - It can use algorithms like **first-in, first-out (FIFO)** or **round-robin**.

Connectors

- The **Sub-dimensions**

Subdimensions provide a more detailed and granular view of each main dimension.

- **Data Exchange Subdimensions:**

- **Data Format:**

- The structure of the data being exchanged, such as **XML**, **JSON**, or a **binary format**.

- **Directionality:**

- Whether data flow is **unidirectional** (one-way) or **bidirectional** (two-way).

- **Synchronization Subdimensions:**

- **Concurrency:**

- How the connector handles multiple simultaneous requests.
 - This can be **single-threaded** or **multi-threaded**.

- **Persistence:** Whether messages are stored if the recipient is unavailable. This can be **transient** (not stored) or **persistent** (stored in a queue or database).

Connectors

- The **Sub-dimensions**

Subdimensions provide a more detailed and granular view of each main dimension.

- **Protocol Subdimensions:**

- **Statefulness:**

- Whether the connector maintains state information across multiple interactions.
 - It can be **stateless** (like HTTP) or **stateful** (like an FTP session).

- **Error Handling:**

- How the connector deals with failures or unexpected events, including **retries** and **timeout mechanisms**.

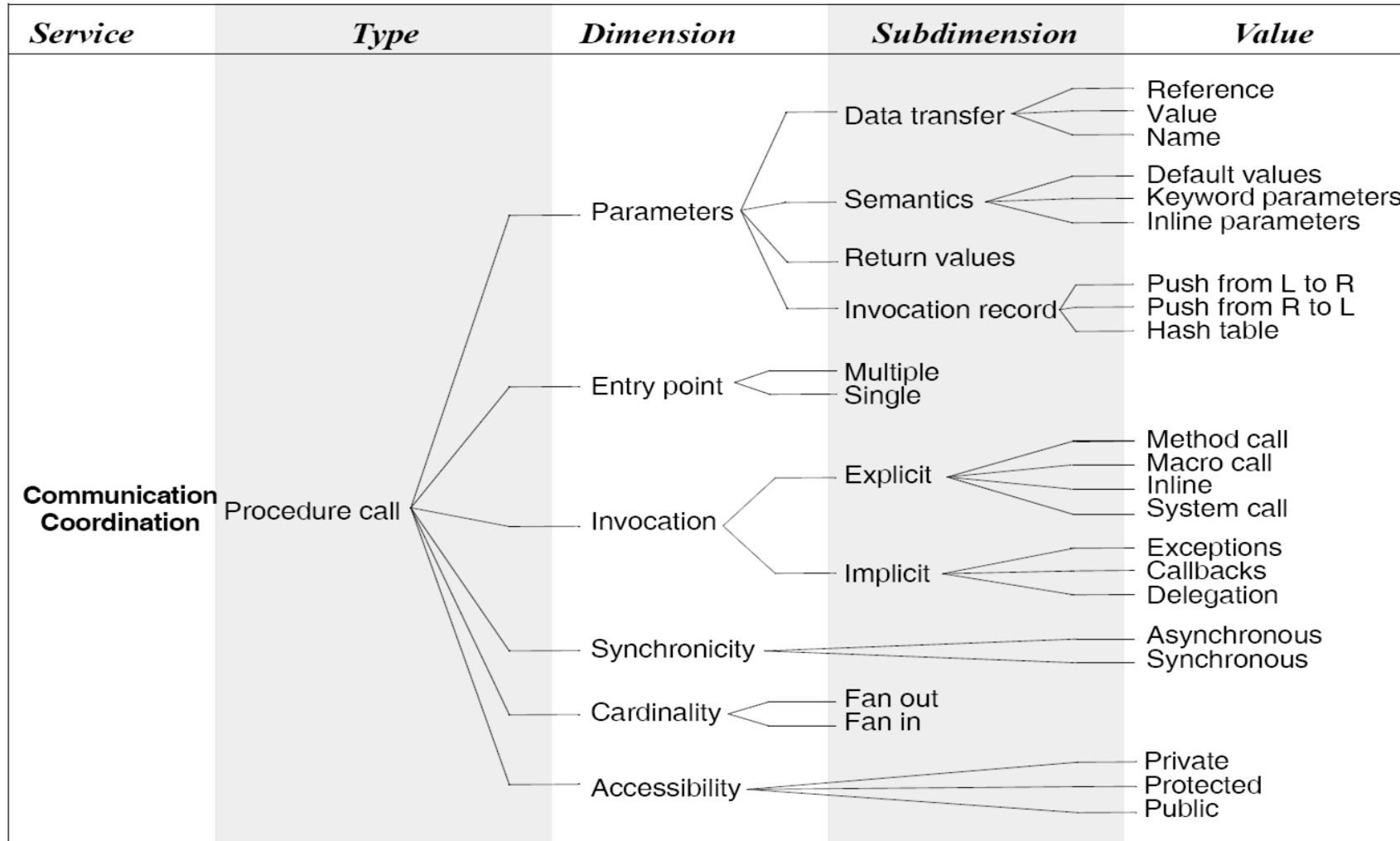
Procedure Call Connectors

- Service(s) provided:
- Dimensions and corresponding sub-dimensions and Values

Procedure Call Connectors

- Sub-dimensions:
 - Data transfer
 - Semantic
- Values:

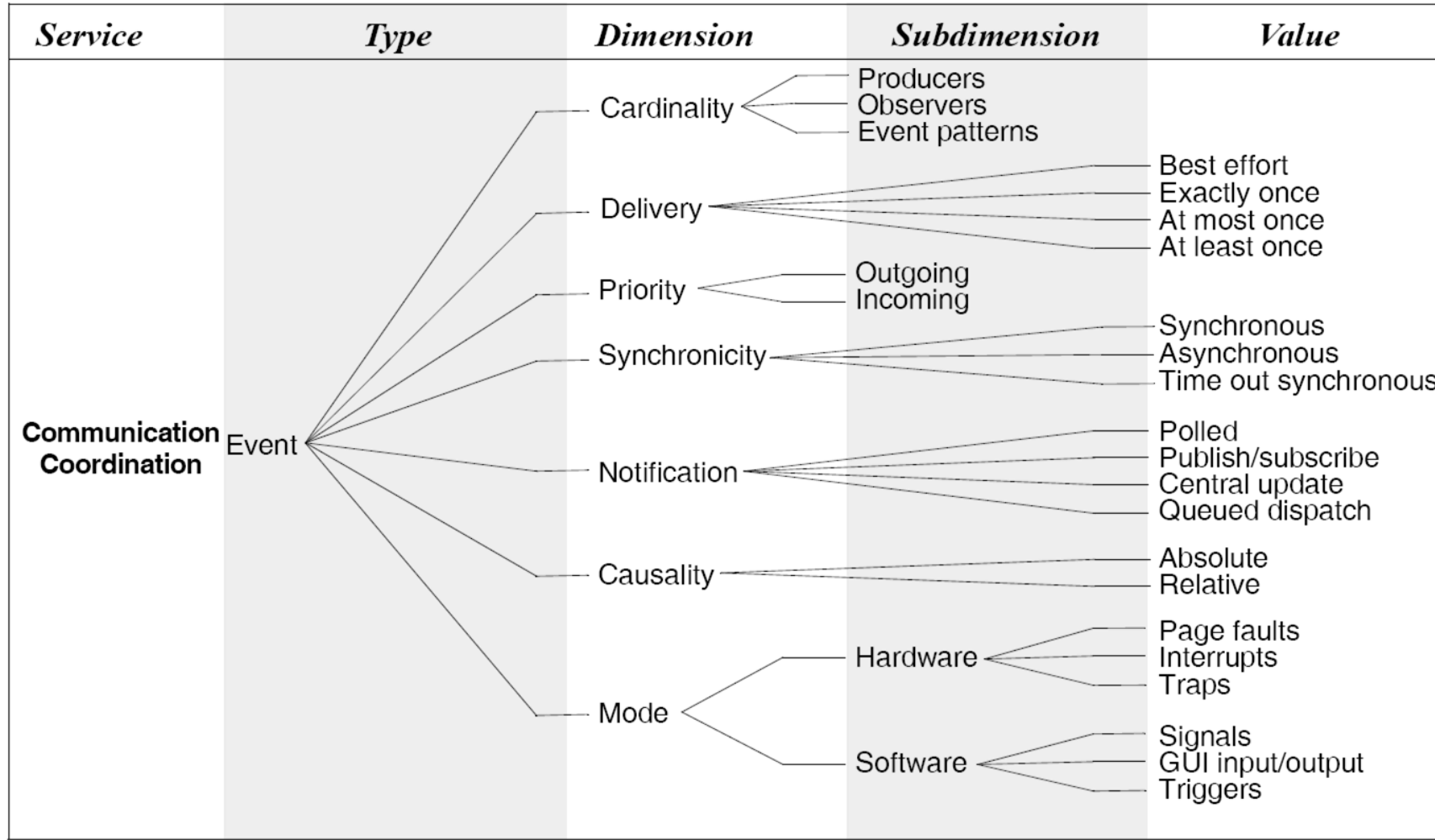
Procedure Call Connectors



Event Connectors

- An event is the instantaneous effect of the normal or abnormal termination of the invocation of an operation on an object, which occurs at that object's location.
- They are **similar to procedure call connectors** as they affect the flow of control among components providing coordination services.
- Flow is participated by an event.
- After recognizing the occurrence of an event, event notification is generated which concludes into component handling the event gaining the control.
- They form **virtual connection** between components interested in the same event topics.
- Typically found in distributed applications that requires asynchronous communication.

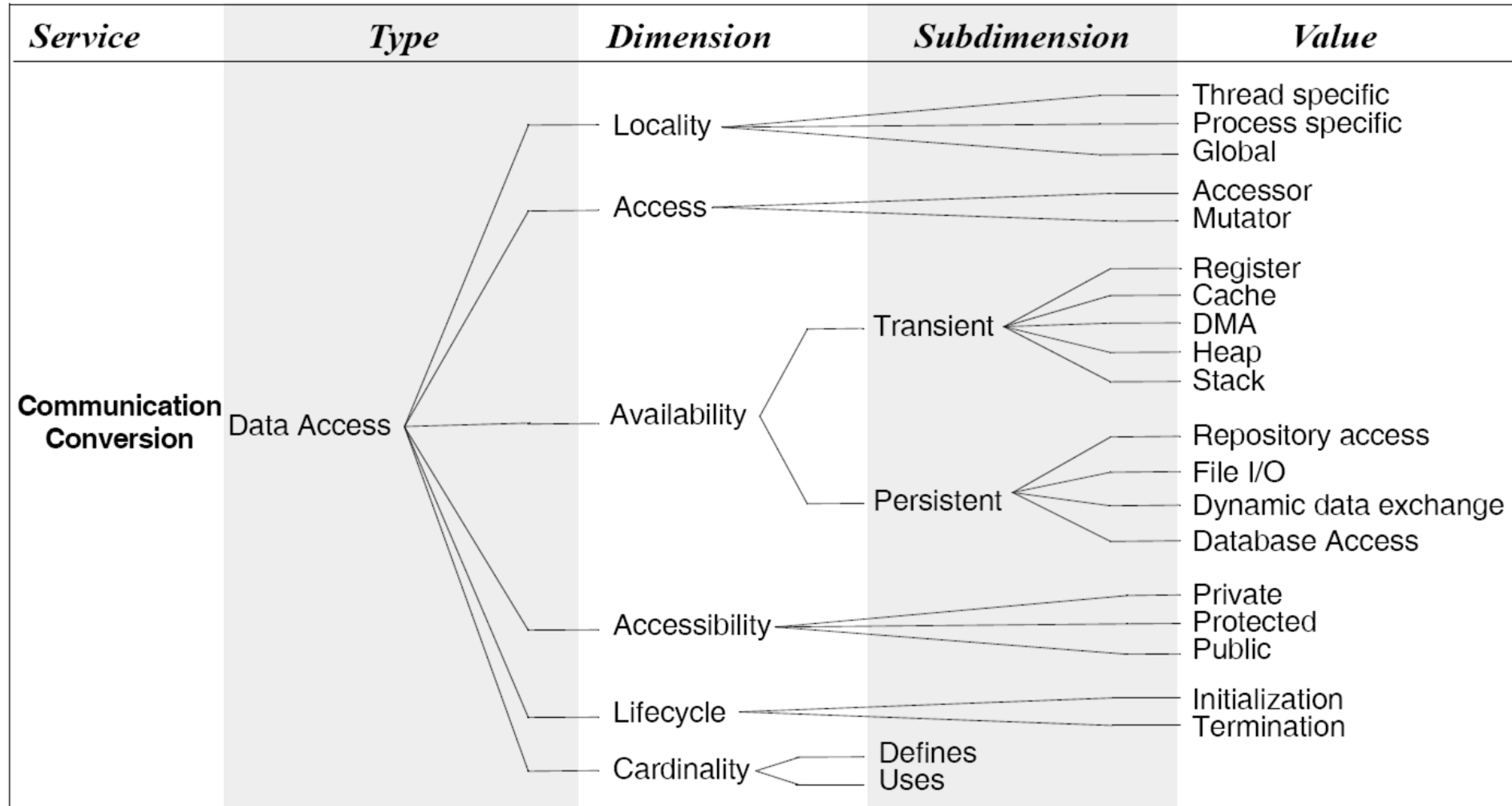
Event Connectors



Data Access Connectors

- Allow components to access data maintained by data store components as a communication service
- Needs preparation of data store before and cleanup after access
- Format may be different before and after access, hence connector may transform data from one format to another
- **Persistent data:** data access including query mechanism, such as SQL for data access, accessing information in software components repositories.
- **Transient data:** data access through heap, stack memory , information caching etc.
- Could enable global access, Mutating (changing data) access

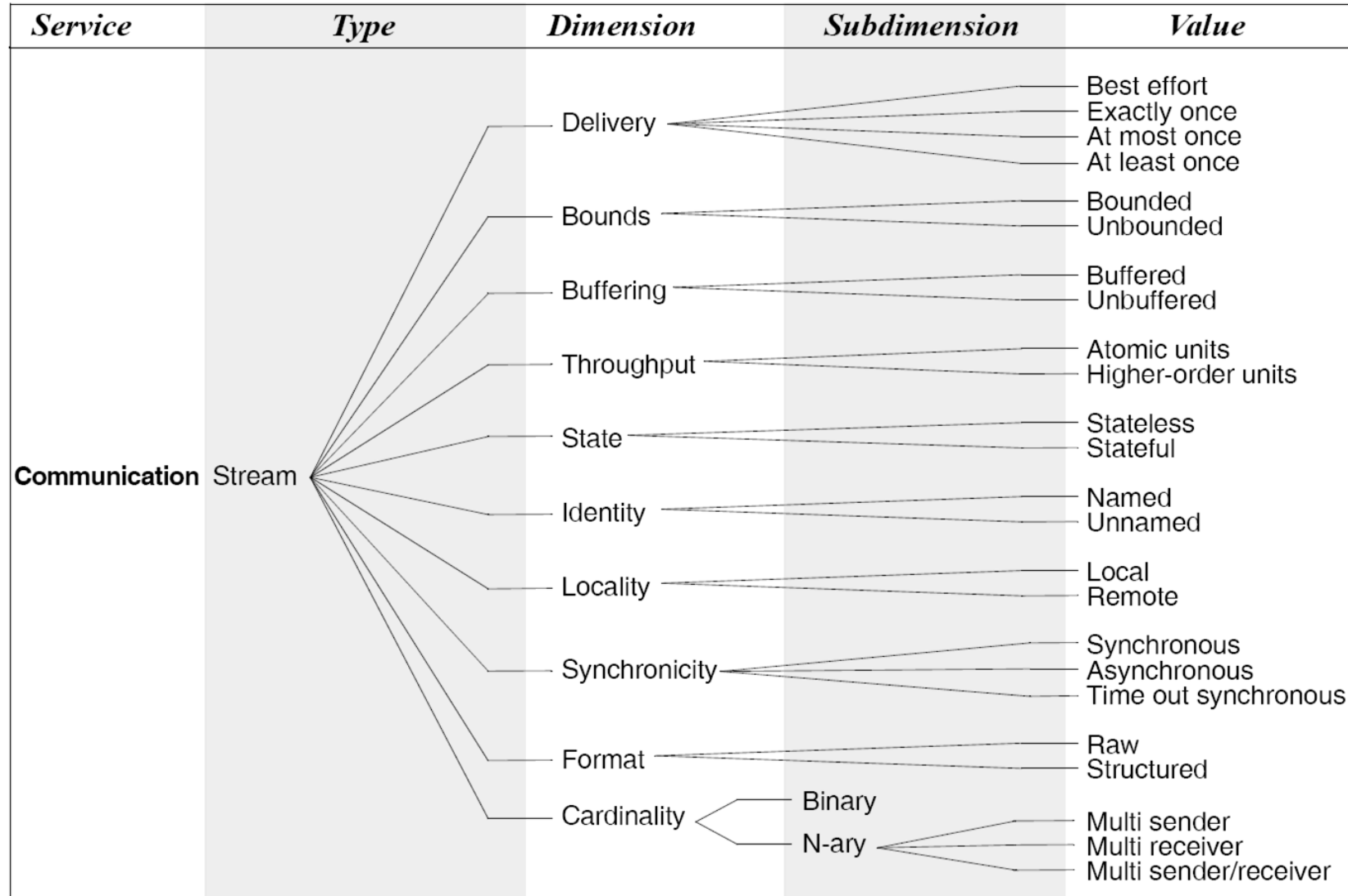
Data Access Connectors



Stream Connectors

- Used for transferring large amount of data between autonomous processes (as a communication service)
- Used in client-server system using data transfer protocols to deliver results of computation
- Can be combined with other types of connectors to provide composite type connectors

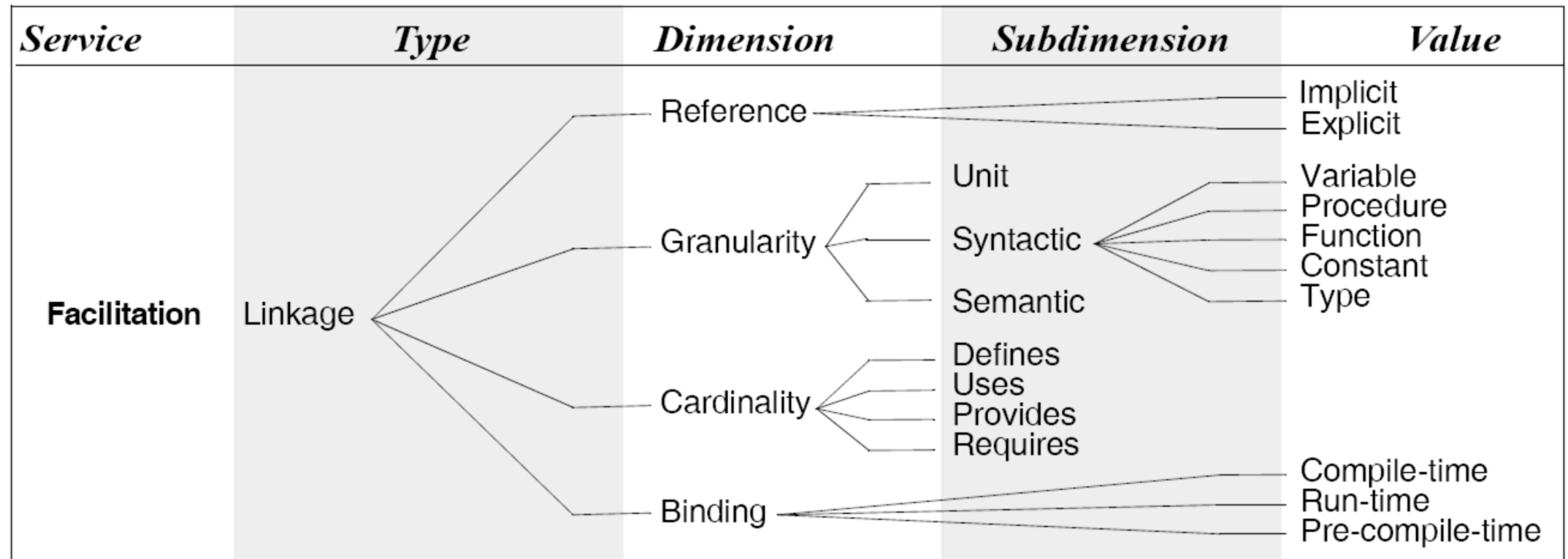
Stream Connectors



Linkage Connectors

- Used to tie the system components together and hold in the same state during operation
- Establishes channels/ ducts for communication and coordination through higher-order connectors to enforce certain semantics.
- Works as facilitations services.
- Used to establish the link and later may disappear
- Semantic interconnections specifies how the linked components are supposed to interact.

Linkage Connectors



Distributor Connectors

- Performs identification of interaction paths and subsequent routing of communication and coordination information among components along these paths.
- Provides facilitation services
- Always works in association with other connectors such as stream or procedural call

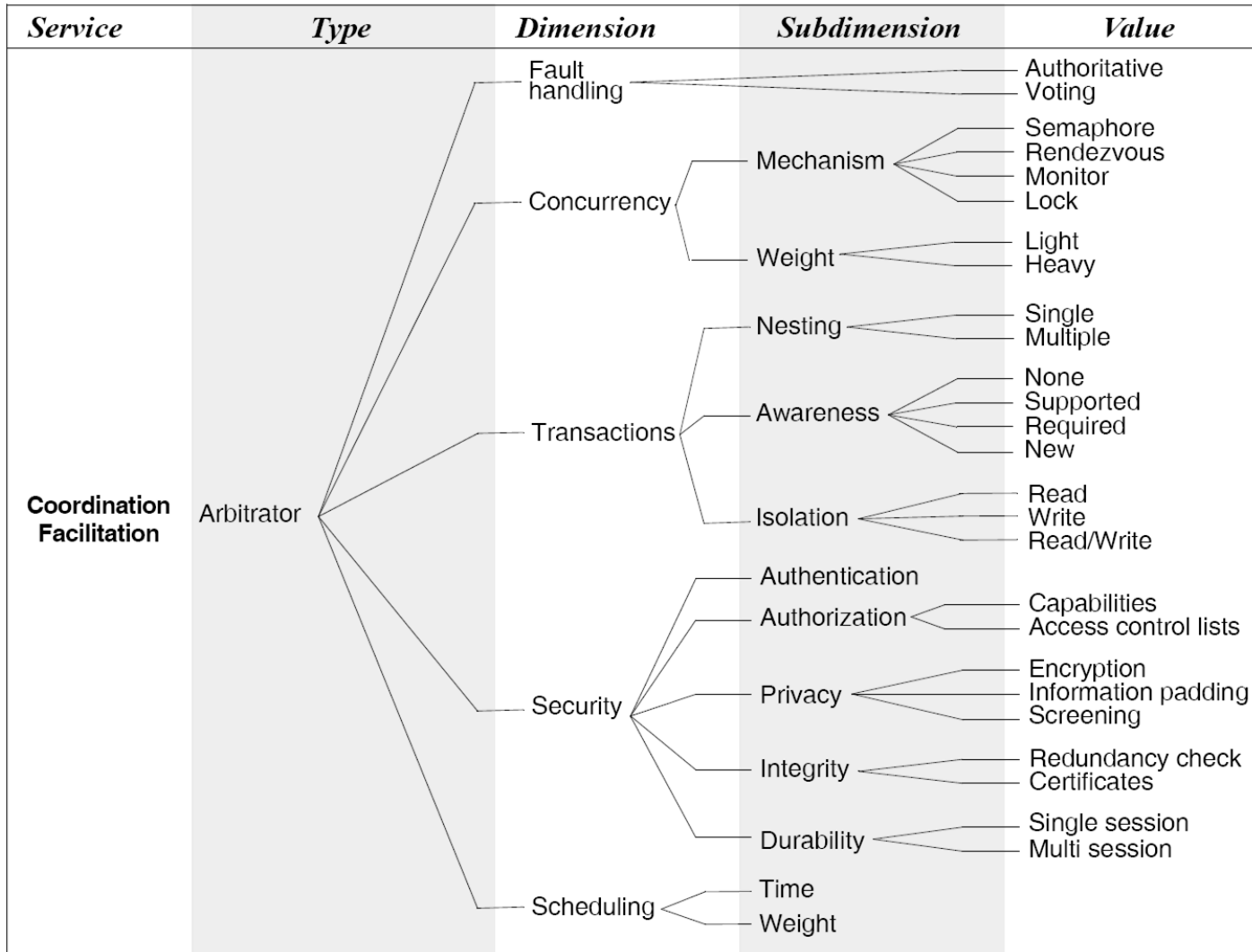
Distributor Connectors

<i>Service</i>	<i>Type</i>	<i>Dimension</i>	<i>Subdimension</i>	<i>Value</i>
Facilitation	Distributor	Naming	Structure based	Hierarchical
				Flat
		Delivery	Attribute based	
			Semantics	Best effort
				Exactly once
				At most once
				At least once
			Mechanism	Unicast
				Multicast
				Broadcast
		Routing	Membership	Bounded
				Ad-hoc
			Path	Static
				Cached
				Dynamic

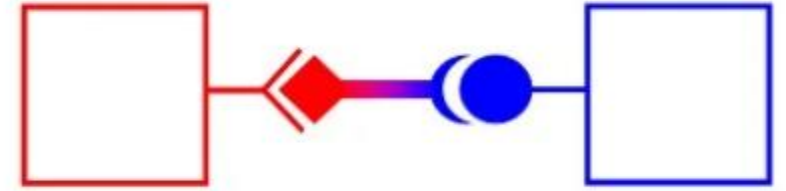
Arbitrator Connectors

- Streamlines system operations provide facilitation by resolving conflicts
- Coordination by redirecting flow of control
- E.g. in multithreaded environment shared memory access is provided through synchronization and concurrency control

Arbitrator Connectors



Adaptor Connectors



- Provide facilities to support interaction between components those may mismatch
- Interoperation in heterogeneous environment such as different programming languages or computing platforms

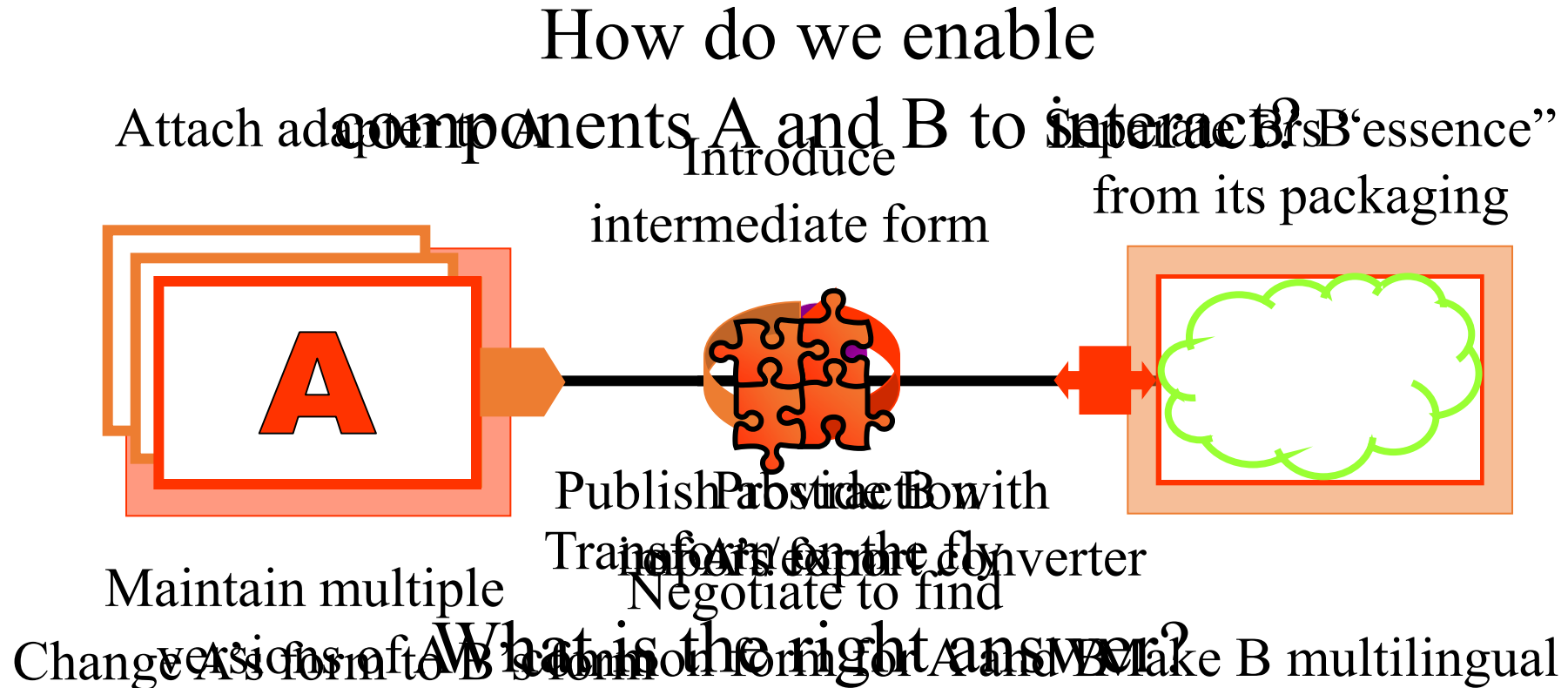
Adaptor Connectors

<i>Service</i>	<i>Type</i>	<i>Dimension</i>	<i>Subdimension</i>	<i>Value</i>
Conversion	Adaptor	Invocation conversion	Address mapping Marshalling Translation	
		Packaging conversion		Wrappers Packagers
		Protocol conversion		
		Presentation conversion		

Merits of Connectors

- Connectors allow modeling of arbitrarily complex interactions
- Connector flexibility aids system evolution
 - Component addition, removal, replacement, reconnection, migration
- Support for connector interchange is desired
 - Aids system evolution
 - May not affect system functionality
- Libraries of OTS connector implementations allow developers to focus on application-specific issues
- Difficulties
 - Rigid connectors
 - Connector “dispersion” in implementations
- Key issue
 - Performance vs. flexibility

Role and Challenge of Software Connectors



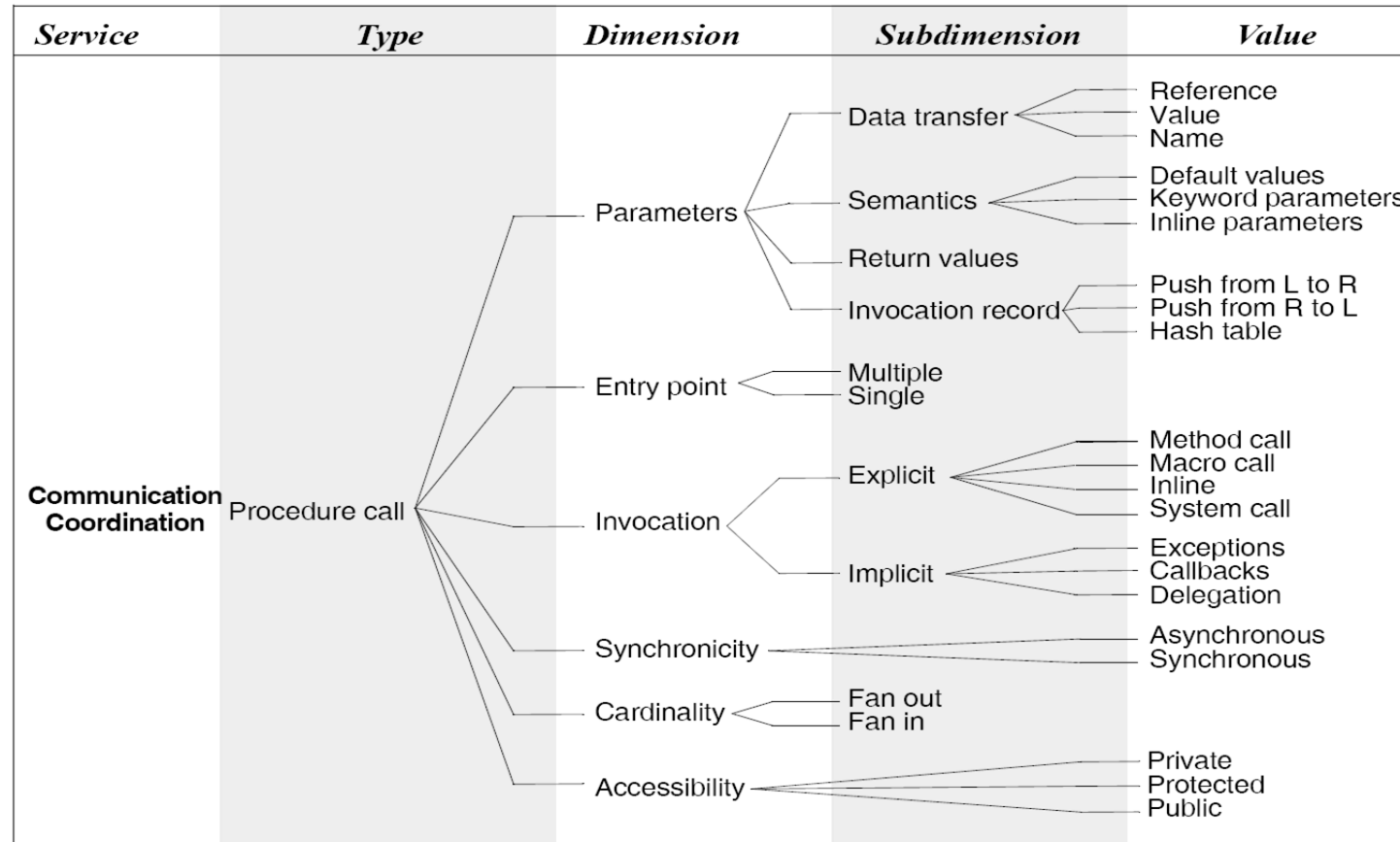
How Does One Select a Connector?

- Determine a system's interconnection and interaction needs
 - Software interconnection models can help
- Determine roles to be fulfilled by the system's connectors
 - Communication, coordination, conversion, facilitation
- For each connector
 - Determine its appropriate type(s)
 - Determine its dimensions of interest
 - Select appropriate values for each dimension
- For multi-type, i.e., composite connectors
 - Determine the atomic connector compatibilities

Simple Example

- System components will execute in two processes on the same host
 - Mostly intra-process
 - Occasionally inter-process
- The interaction among the components is synchronous
- The components are primarily computation-intensive
 - There are some data storage needs, but those are secondary
- Select procedure call connectors for intra-process interaction
- Combine procedure call connectors with distributor connectors for inter-process interaction
 - RPC
- Select the values for the different connector dimensions
 - What are the appropriate values?
 - What values are imposed by your favorite programming language(s)?

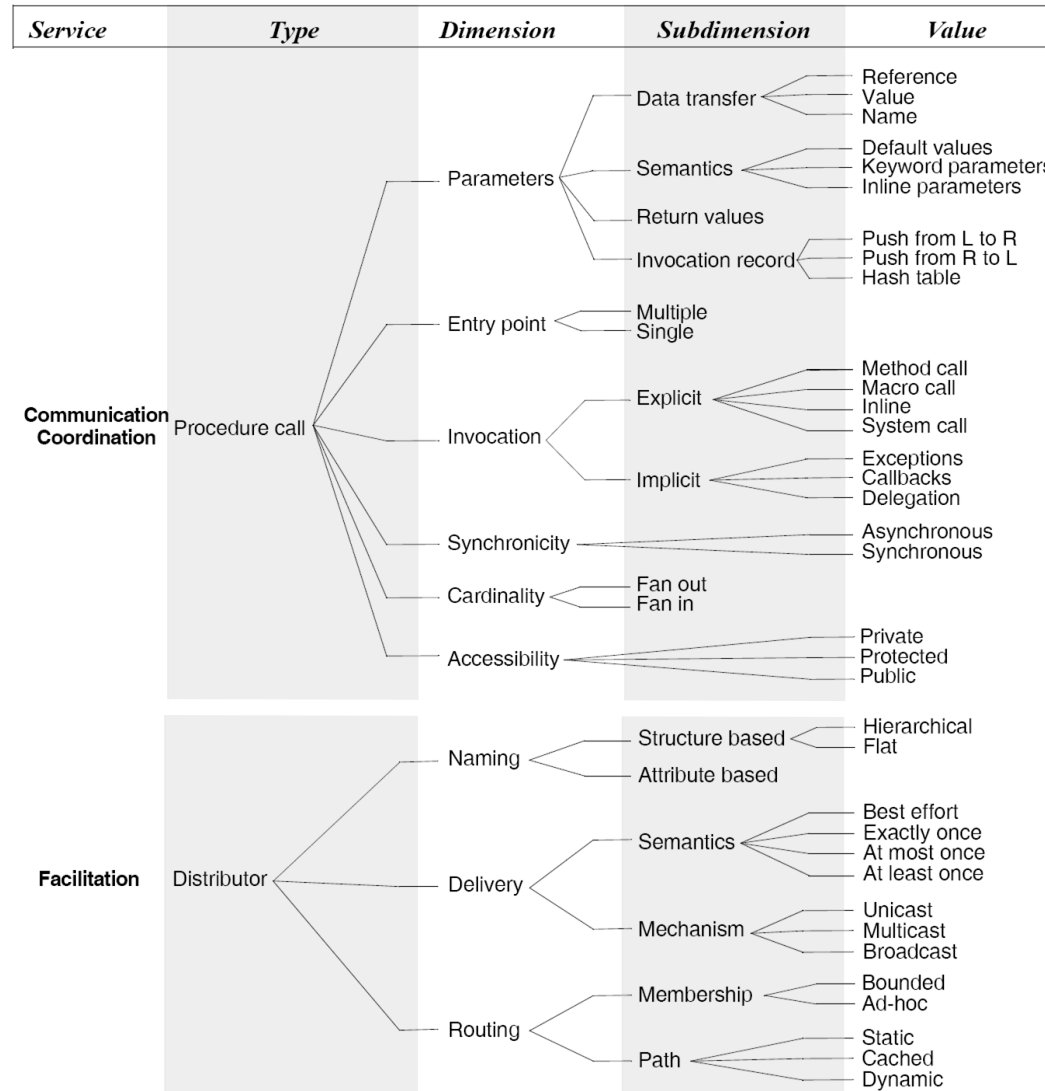
Procedure Call Connectors Revisited



Distributor Connectors Revisited

<i>Service</i>	<i>Type</i>	<i>Dimension</i>	<i>Subdimension</i>	<i>Value</i>
Facilitation	Distributor	Naming	Structure based	Hierarchical
				Flat
		Delivery	Attribute based	
			Semantics	Best effort
				Exactly once
				At most once
				At least once
			Mechanism	Unicast
				Multicast
				Broadcast
		Routing	Membership	Bounded
				Ad-hoc
			Path	Static
				Cached
				Dynamic

Two Connector Types in Tandem



Select the appropriate values for PC and RPC!

Selecting appropriate type of Connector

1. Select the specific set of interacting components.
2. Determine the interaction services the components need.
 - a) Identify specific characteristics of the components' interaction
 - b) Study components' architectural descriptions, implementation language and/ or framework
3. Based on the identified interaction services, determine a subset of connector types that comprise the initial candidate set for providing those services.
4. Evaluate each connectors type from the chosen subset based on the details of interaction requirements
5. For each of the remaining candidate connector types, set the values for the necessary dimensions and subdimensions as appropriate

Connector Dimension Inter-Relationships

- Requires – 
 - Choice of one dimension mandates the choice of another
- Prohibits – 
 - Two dimensions can never be composed into a single connector
- Restricts – 
 - Dimensions are not always required to be used together
 - Certain dimension combinations may be invalid
- Cautions – 
 - Combinations may result in unstable or unreliable connectors

Dimension Inter-Relationships

[illegible]

Well Known Composite Connectors

- Grid connectors (e.g., Globus)
 - Procedure call
 - Data access
 - Stream
 - Distributor
- Peer-to-peer connectors (e.g., Bittorrent)
 - Arbitrator
 - Data access
 - Stream
 - Distributor
- Client-server connectors
- Event-based connectors